

P0217R0: Proposed wording for structured bindings

Introduction

This paper presents the proposed wording to implement structured bindings as described by Herb Sutter, Bjarne Stroustrup, and Gabriel dos Reis in P0144R1.

Wording

In section 6.5 [stmt.iter] paragraph 1, change the grammar to allow a decomposition declaration:

```
for-range-declaration:  
    attribute-specifier-seqopt decl-specifier-seq declarator  
    attribute-specifier-seqopt decl-specifier-seq &opt { identifier-list }
```

In section 7 [dcl.dcl] paragraph 1, change the grammar to allow a decomposition declaration:

```
simple-declaration:  
    decl-specifier-seq init-declarator-listopt ;  
    attribute-specifier-seq decl-specifier-seq init-declarator-list ;  
    attribute-specifier-seqopt decl-specifier-seq &opt { identifier-list } brace-or-equal-initializer ;
```

Add a new paragraph after 7 [dcl.dcl] paragraph 8:

A *simple-declaration* with an *identifier-list* is called a *decomposition declaration*. The *decl-specifier-seq* shall contain only the *type-specifier* `auto` (7.1.6.4 [dcl.spec.auto]) and *cv-qualifiers*. The *brace-or-equal-initializer* shall be of the form "`= assignment-expression`" or of the form "`{ assignment-expression }`". If the *assignment-expression* is a prvalue, the *decl-specifier-seq* does not contain the *cv-qualifier* `const`, and `&` follows the *decl-specifier-seq*, the program is ill-formed.

Add after 7.1.6.4 [dcl.spec.auto] paragraph 8:

A decomposition declaration introduces the *identifiers* of the *identifier-list* as variable names in the order of appearance, with types deduced from the initializer. If the *assignment-expression* of the decomposition declaration has scalar type, the *identifier-list* shall contain a single *identifier* and a single variable is introduced as if by

```
attribute-specifier-seqopt decl-specifier-seq &opt identifier brace-or-equal-initializer ;
```

If the *assignment-expression* has union type, the program is ill-formed.

Otherwise, i.e. the *assignment-expression* has non-union class or array type E , let e be a glvalue of type E .

- If the *assignment-expression* is a prvalue, e is an xvalue that designates a temporary object copy-initialized from the *assignment-expression*. The lifetime of the temporary ends immediately after the last variable declaration (see below) if no `&` follows the *decl-specifier-seq* in the decomposition declaration, and is extended as usual (12.2 [class.temporary]) otherwise.
- Otherwise, e refers to the result of evaluating the *assignment-expression* and has its value category.

The expression get_n is defined as follows:

- If E is an array type, get_n is $e[n]$.
- If E is a class type, the *unqualified-id* `get` is looked up in the scope of E as if by class member access lookup (3.4.5 [basic.lookup.classref]), and if that finds at least one declaration, get_n is $e.get<n>()$.
- Otherwise, get_n is $get<n>(e)$, where `get` is looked up in the associated namespaces (3.4.2 [basic.lookup.argdep]). [Note: Ordinary unqualified lookup (3.4.1 [basic.lookup.unqual]) is not performed. -- end note]

If E is an array type, the number of *identifiers* shall be equal to the number of elements of E . Otherwise, if the expression get_0 is not well-formed, all of E 's non-static data members and bit-fields shall be public direct

members of E or of the same public base class of E , E shall not have an anonymous union member, and the number of *identifiers* in the *identifier-list* shall be equal to the number of non-static data members of E .

For the i -th *identifier* v_i in the *identifier-list*, where numbering starts at 1, a variable is declared using the *attribute-specifier-seq* and *decl-specifier-seq* from the decomposition declaration, and a *declarator* $\&_{\text{opt}} v_i$, where $\&$ is present if and only if it is present in the decomposition declaration. If the expression get_0 is well-formed, the variable is initialized with the expression get_{i-1} , otherwise it is initialized with the expression $e.m_i$, where m_i designates the i -th non-static data member of E . The variable is copy-initialized or direct-initialized as indicated by the form of the *brace-or-equal-initializer* (8.5 [dcl.init]). [Note: The type of v_i is deduced from its initializer as described in the remainder of this section. -- end note] [Example:

```
struct S { int x; volatile double y; };
const S f();
auto { x, y } = f();
```

The type of x is "int", the type of y is "double". -- end example]